Task 1 (AWS Cafe Challenge Portfolio)

Challenge 3 - Migrating a Database to Amazon RDS

Academic Year	Module	Assessment Number	Assessment Type
2026	5CS048 - Cloud Systems	Task 1 Challenge 3	Portfolio

Student ID: 2462250

Student Name: Praju Maharjan

Group: L5CG9

Module Leader: Mr. Deepson Shrestha

Tutor: Mr. Prabin Sapkota

Submitted On: 12 March 2026

Table of Contents

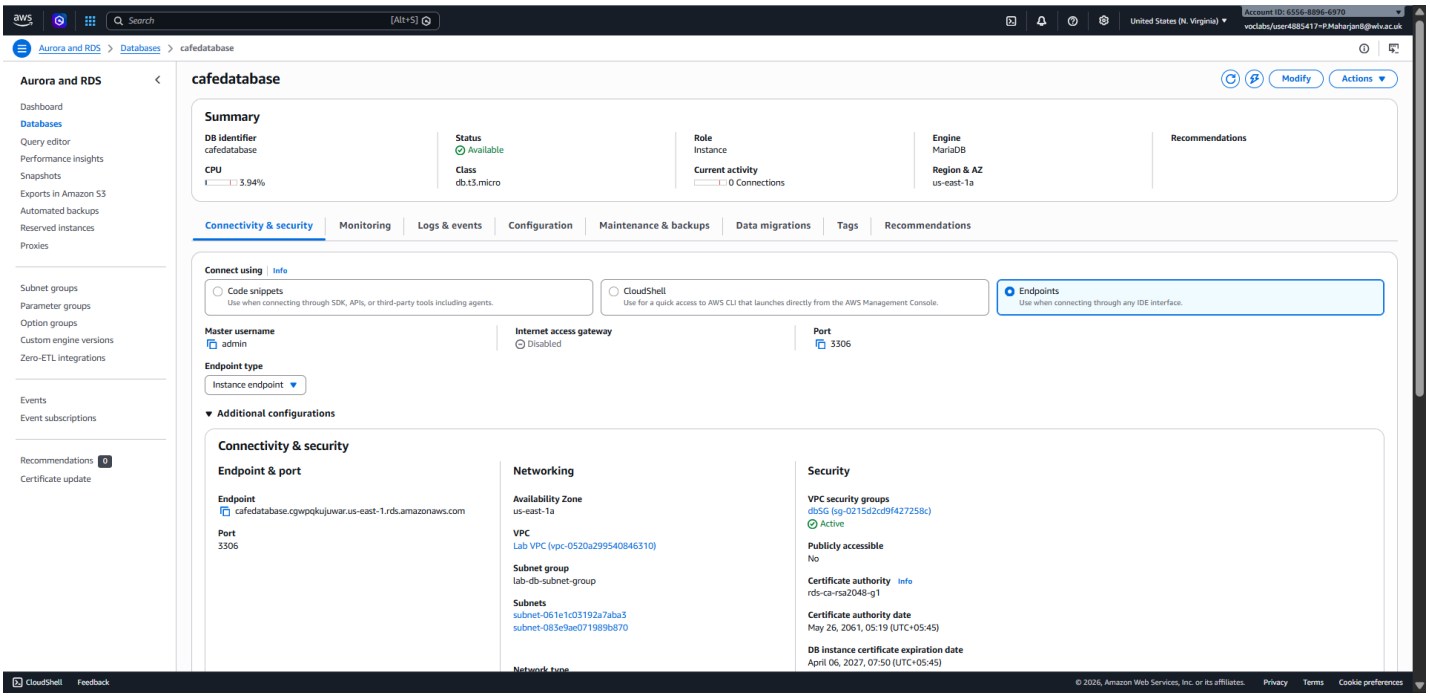
Challenge (Cafe) lab: Migrating a Database to Amazon RDS

Challenge 1: Creating an RDS instance for the café application

Task 1: Creating an RDS instance

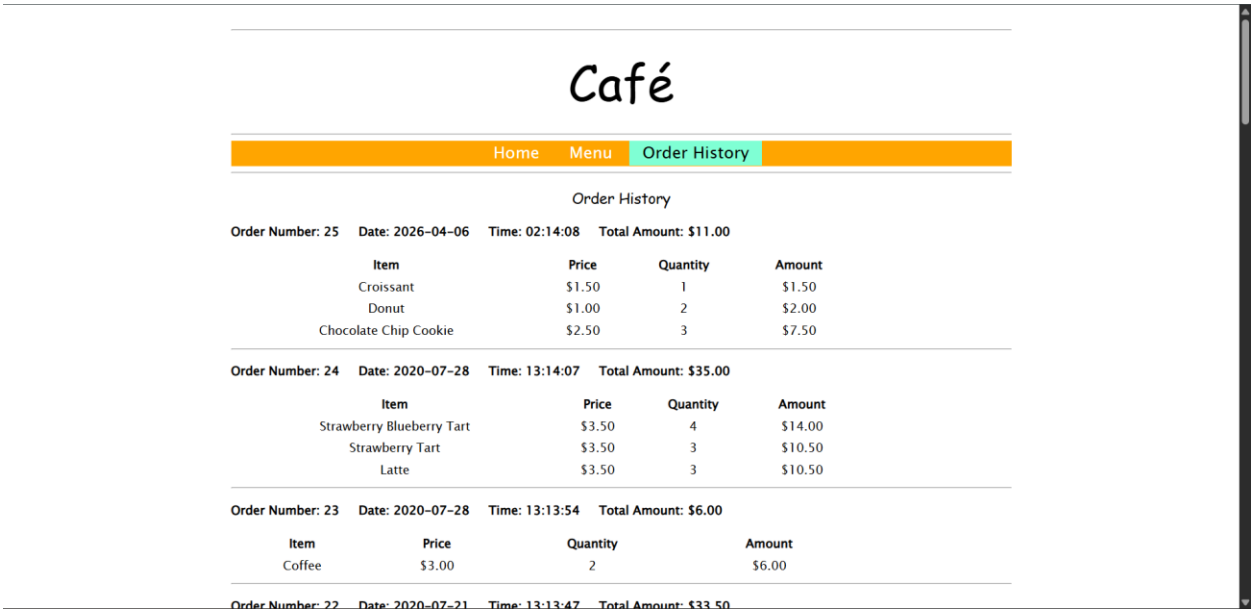
Show a screenshot of the Amazon RDS console displaying the CafeDatabase instance with:

- **Status: Available**
- **Engine: MariaDB 10.6.25**
- **DB instance class: db.t3.micro**
- **The RDS endpoint URL visible in the Connectivity & security tab**



Task 2: Analyzing the existing café application deployment

Show a screenshot of the café application Order History page (http://<public-ip>/cafe) in the browser, displaying the full list of historical customer orders that were present before migration.



Challenge 2: Exporting data from the old database and establishing a connection to the new database

Task 3: Working with the database on the EC2 instance

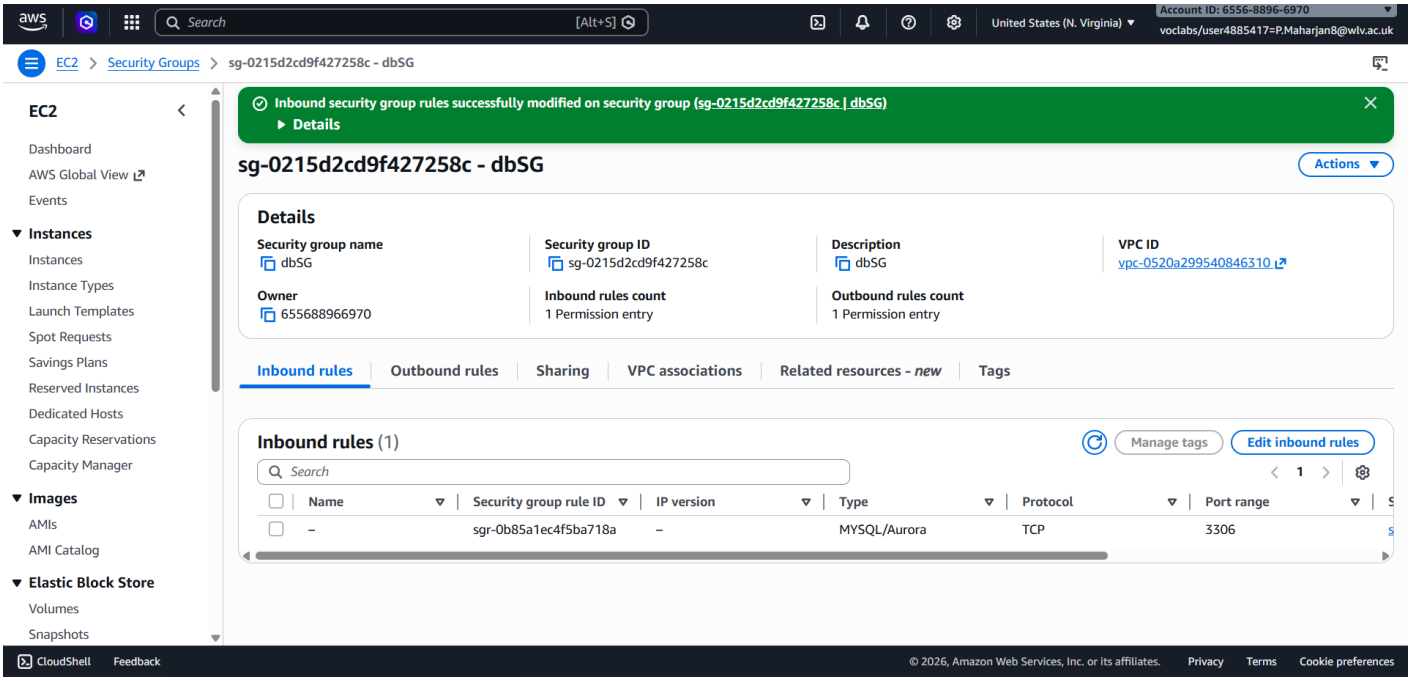
Show a screenshot of the Systems Manager Session Manager terminal with the output of the ls command in the /home/ec2-user/ directory, confirming that the CafeDbDump.sql file was successfully created.

```
[ec2-user@cafeserver ~]$ ls
CafeDbDump.sql
[ec2-user@cafeserver ~]$
```

Task 4: Working with the RDS database

Show two screenshots:

- **The EC2 console showing the inbound rules of the dbSG security group, with a rule permitting TCP port 3306 sourced from the CafeServer EC2 instance's security group (not open to all IPs).**



- **The Systems Manager Session Manager terminal showing the output of show databases; after successfully connecting to the RDS instance with the mysql -u**

admin command, confirming a live connection to the RDS endpoint.

```
MariaDB [(none)]> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| innodb             |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
5 rows in set (0.00 sec)

MariaDB [(none)]>
```

Challenge 3: Importing data and connecting the application to the new database

Task 5: Importing the data into the RDS database instance

Show a screenshot of the Systems Manager Session Manager terminal displaying the output of the following commands run against the RDS instance:

- **show databases; — with cafe_db visible in the list**

```
MariaDB [cafe_db]> show databases;
+--------------------+
| Database           |
+--------------------+
| cafe_db            |
| information_schema |
| innodb             |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
6 rows in set (0.00 sec)
```

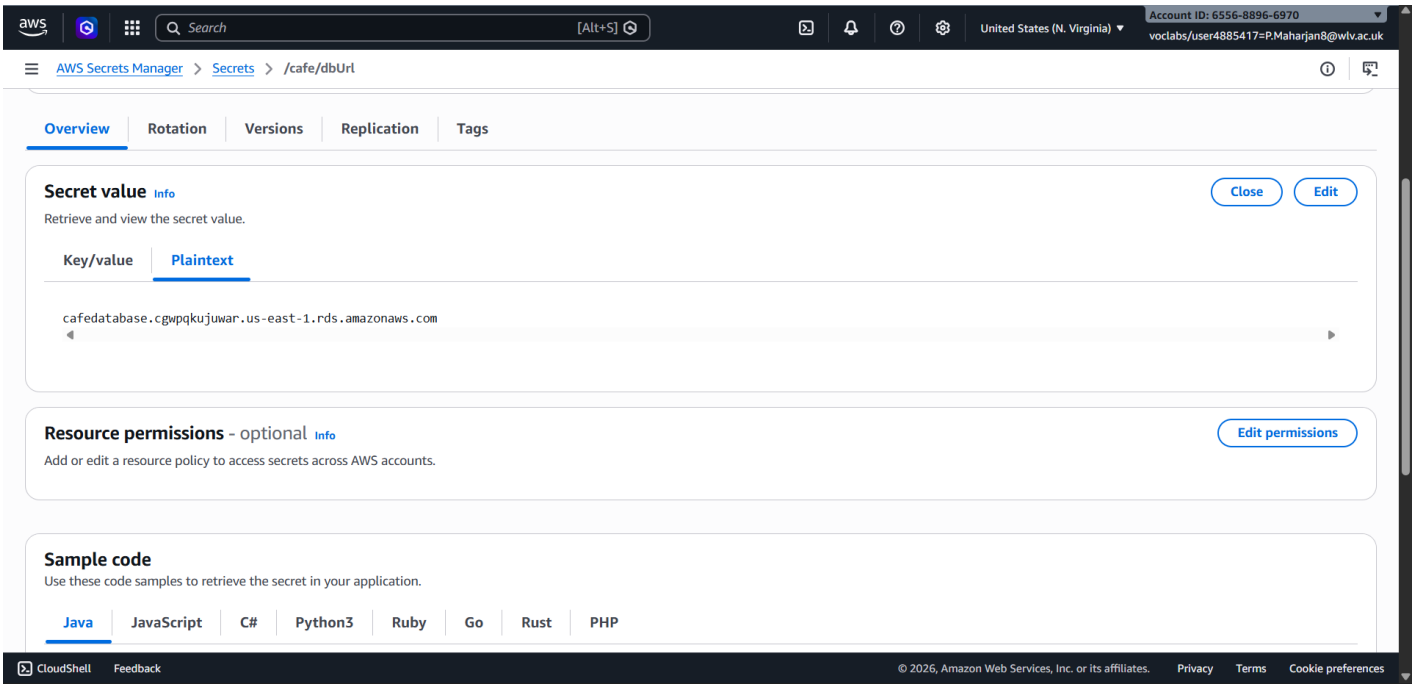
- **select * from `order`; — showing at least 10 migrated orders**

```
MariaDB [cafe_db]> select * from `order`;
+--------------+---------------------+--------+
| order_number | order_date_time     | amount |
+--------------+---------------------+--------+
|            1 | 2020-06-20 13:09:07 |  20.00 |
|            2 | 2020-06-21 13:09:23 |  24.00 |
|            3 | 2020-06-22 13:09:38 |  32.50 |
|            4 | 2020-06-22 13:09:49 |  10.50 |
|            5 | 2020-06-24 13:10:02 |  42.00 |
|            6 | 2020-06-25 13:10:20 |  14.00 |
|            7 | 2020-06-25 13:10:30 |  35.00 |
|            8 | 2020-06-27 13:10:36 |   6.00 |
|            9 | 2020-06-28 13:10:46 |  18.00 |
|           10 | 2020-07-02 13:10:58 |  18.00 |
|           11 | 2020-07-03 13:11:08 |  19.50 |
|           12 | 2020-07-04 13:11:17 |   9.00 |
|           13 | 2020-07-05 13:11:27 |  26.50 |
|           14 | 2020-07-05 13:11:40 |  42.00 |
|           15 | 2020-07-06 13:12:27 |  35.00 |
|           16 | 2020-07-09 13:12:35 |  19.50 |
|           17 | 2020-07-12 13:12:48 |  57.50 |
|           18 | 2020-07-14 13:13:04 |  34.00 |
|           19 | 2020-07-15 13:13:17 |  29.00 |
|           20 | 2020-07-18 13:13:27 |  14.00 |
|           21 | 2020-07-20 13:13:36 |  17.50 |
|           22 | 2020-07-21 13:13:47 |  33.50 |
|           23 | 2020-07-28 13:13:54 |   6.00 |
|           24 | 2020-07-28 13:14:07 |  35.00 |
|           25 | 2026-04-06 02:14:08 |  11.00 |
+--------------+---------------------+--------+
25 rows in set (0.00 sec)
```

Task 6: Connecting the café application to the new database

Show two screenshots:

- **The AWS Secrets Manager console showing the /cafe/dbUrl secret with its retrieved value updated to the RDS endpoint (confirming the application has been redirected to the new database).**



- **The café Order History page (http://<public-ip>/cafe) loading successfully in the browser — taken after running sudo service mariadb stop in the terminal — proving the application is now fully powered by the RDS database and no longer dependent on the local EC2 database.**

Café

Home Menu Order History

Order History

Order Number: 25 Date: 2026-04-06 Time: 02:14:08 Total Amount: $11.00

Item	Price	Quantity	Amount
Croissant	$1.50	1	$1.50
Donut	$1.00	2	$2.00
Chocolate Chip Cookie	$2.50	3	$7.50

Order Number: 24 Date: 2020-07-28 Time: 13:14:07 Total Amount: $35.00

Item	Price	Quantity	Amount
Strawberry Blueberry Tart	$3.50	4	$14.00
Strawberry Tart	$3.50	3	$10.50
Latte	$3.50	3	$10.50

Order Number: 23 Date: 2020-07-28 Time: 13:13:54 Total Amount: $6.00

Item	Price	Quantity	Amount
Coffee	$3.00	2	$6.00

Order Number: 22 Date: 2020-07-21 Time: 13:13:47 Total Amount: $33.50

Description

Write in your own wording and clearly cover all of the following (about 250 to 450 words):

1. what you built and what you did in that lab
2. one security control you used and why it matters
3. one performance or reliability feature that is present and how it helps
4. one issue you had and how you fixed it (or one likely issue and how you would fix it)

Ans :

In this lab, I migrated the café application's database from a locally hosted MariaDB instance on an EC2 server to a managed Amazon RDS database. First, I created an RDS instance using MariaDB with the required configurations such as instance type, storage, and private network setup. Then, I analyzed the existing café application and confirmed that it was working correctly with the current database by viewing order history. After that, I exported the existing database using the mysqldump utility and saved it as a SQL file. I then established a secure connection between the EC2 instance and the RDS database by configuring the appropriate security group rules. Once the connection was successful, I imported the data into the RDS instance and verified that all tables and order records were correctly migrated. Finally, I updated the application configuration through AWS Secrets Manager so that the web application connects to the new RDS database instead of the local database, and confirmed this by stopping the local database while the application continued to function normally.

One important security control used in this lab was the configuration of the RDS security group to allow access only from the EC2 instance's security group instead of opening the database to all IP addresses. This ensures that only authorized resources can connect to the database, reducing the risk of unauthorized access or attacks. This control is important because databases often contain sensitive information such as customer orders, and restricting access helps maintain data confidentiality and integrity.

A key reliability feature in this setup is the use of Amazon RDS as a managed database service. RDS provides automated backups, patching, and maintenance, which reduces the risk of data loss and system downtime. This improves reliability because the database is no longer dependent on a single EC2 instance, and AWS handles critical management tasks, allowing the application to run more consistently and efficiently.

One issue that could occur during this lab is the inability to connect to the RDS instance from the EC2 instance. This is usually caused by incorrect security group configurations. I resolved (or would resolve) this by ensuring that port 3306 is open only to the EC2 instance's security group and verifying connectivity using tools like nmap. Additionally, checking credentials and endpoint configuration helps ensure successful database connections.